

Edge - Spring Boot 공유 저장소/DB 전송 큐 계약

1. 목적

Edge가 생성한 최종 보고서 ZIP을 Spring Boot에 HTTP multipart로 다시 보내지 않는다. Edge와 Spring Boot가 함께 접근할 수 있는 공유 저장소에 ZIP을 한 번만 보관하고, MariaDB 전송 큐에는 파일의 **논리 저장소 키와 상대 경로**만 등록한다.

Spring Boot는 스케줄러로 큐를 조회한 뒤 공유 저장소의 파일을 직접 읽어 LIMS에 전송한다. 따라서 Spring Boot가 ZIP을 별도로 수신하거나 중간 폴더로 복사하는 API는 필요하지 않다.



분석용 report.json, LLM 요청/응답 로그 등 Edge 내부 데이터는 전송 큐에 등록하지 않는다. 사용자에게 전달할 최종 report-package.zip 만 대상이다.

2. 책임 분리

구성요소	책임
실험 PC/C#	Edge 작업 생성, raw 업로드, 업로드 완료, 보고서 생성 요청 및 상태 조회
Edge API/worker	보고서 생성, 공유 저장소에 최종 ZIP 확정, 크기/SHA-256/ZIP 검증, DB 큐 등록
MariaDB	보고서 위치·무결성 메타데이터, 전송 상태, 시도 이력 보관
Spring Boot scheduler	큐 선점, 공유 ZIP 재검증, LIMS 전송, 성공·재시도·실패 상태 기록
LIMS	최종 보고서 수신 및 업무 처리

Spring Boot는 Edge의 `jobs.status`를 변경하지 않는다. Edge 작업 상태와 LIMS 전송 상태는 서로 다른 생명주기로 관리한다.

3. 공유 저장소 설정

Edge 설정:

```
RIST_STORAGE_ROOT=/mnt/rist/reports
RIST_REPORT_STORAGE_KEY=RIST_REPORTS
RIST_REPORT_TRANSFER_MAX_ATTEMPTS=5
```

Spring Boot 설정 예시:

```
rist:
  report-storage:
    roots:
      RIST_REPORTS: /mnt/rist/reports
  report-transfer:
    scheduler-enabled: true
    batch-size: 10
    lease-seconds: 300
```

두 프로세스에서 실제 마운트 경로가 달라도 된다. 동일한 `storage_key=RIST_REPORTS`를 각 프로세스의 로컬 마운트 루트에 매핑하면 된다.

DB storage_key	RIST_REPORTS
DB package_relative_path	jobs/abc/report/report-package.zip
Edge 실제 경로	/mnt/rist/reports/jobs/abc/report/report-package.zip
Spring 실제 경로	D:\rist-share\jobs\abc\report\report-package.zip

DB에는 OS 종속 절대 경로, `file://` URI, ZIP BLOB을 저장하지 않는다. `package_relative_path`는 `/` 구분자를 사용하는 POSIX 상대 경로다.

4. 테이블 구성

기존 `jobs`를 포함해 업무 흐름은 네 테이블로 관리한다. 설치 DDL은 `edge_api_server/deploy/mariadb_report_queue.sql`에 있다.

4.1 jobs

실험 PC 업로드와 Edge 보고서 생성 작업이다. 기존 테이블을 그대로 사용한다.

4.2 report_runs

완성된 보고서 패키지 한 건의 불변에 가까운 메타데이터다.

주요 컬럼	설명
report_id	보고서 ID. 일반 작업은 job_id 와 동일
source_job_id	원본 Edge 작업 ID. 웹 미리보기 보고서는 NULL 가능
request_number	의뢰번호
experiment_code	실험코드
equipment_code	실험장비 코드
operator_id	실험자/작업자 식별자
generation_status	현재 READY 사용
storage_key	공유 저장소 논리 키
package_relative_path	저장소 루트 기준 ZIP 상대 경로
package_size_bytes	ZIP 크기
package_sha256	ZIP SHA-256 소문자 64자리
generated_at	Edge 보고서 생성 시각

4.3 report_transfers

Spring Boot가 소비하는 LIMS 전송 큐다. 보고서·목적지 조합은 한 건만 존재한다.

상태:

상태	의미
PENDING	전송 대기
PROCESSING	특정 Spring worker가 lease를 획득해 처리 중
RETRY_WAIT	일시 오류 후 next_retry_at 까지 대기
COMPLETED	LIMS 전달 완료
FAILED	비재시도 오류 또는 최대 시도 횟수 초과
CANCELLED	운영자가 전송 취소

request_number, experiment_code, equipment_code, operator_id 는 스케줄러가 큐를 조회할 때 불필요한 조인을 줄이고 감사 시점의 값을 보존하기 위해 큐에도 기록한다.

4.4 report_transfer_attempts

실제 LIMS 호출 한 번마다 한 행을 남긴다. 응답 코드, 오류 코드, 시작·종료 시각, worker ID를 기록하며 대용량 응답 본문이나 ZIP은 저장하지 않는다.

5. Edge 등록 트랜잭션

1. 보고서 렌더링과 ZIP 패키징을 완료한다.
2. 최종 ZIP이 공유 저장소 루트 내부의 일반 파일인지 확인한다.
3. ZIP 구조를 열어 유효성을 확인한다.
4. 파일 크기와 SHA-256을 계산한다.
5. 하나의 DB 트랜잭션에서 `report_runs` 를 등록하고 `report_transfers(status=PENDING)` 를 등록한다.
6. 큐 등록까지 성공한 후 `jobs.status=COMPLETED` 로 전환한다.

같은 보고서에 대한 재등록은 `report_id + destination` 과 `idempotency_key` 의 유니크 키로 중복 큐 생성을 막는다. 큐 등록 실패 시 Edge 작업은 `FAILED` 가 되며 오류 코드는 `REPORT_QUEUE_REGISTRATION_FAILED` 다.

`jobs.COMPLETED` 의 의미는 **Edge 보고서 생성과 DB 큐 등록 완료**다. LIMS 전송 완료 여부는 반드시 `report_transfers.status` 로 판단한다.

6. Spring Boot 스케줄러 동작

6.1 작업 선점

여러 Spring 인스턴스가 동시에 실행될 수 있으므로 짧은 DB 트랜잭션에서 `FOR UPDATE SKIP LOCKED` 로 한 건을 선점한다.

```
START TRANSACTION;

SELECT transfer_id
FROM report_transfers
WHERE attempt_count < max_attempts
  AND (
    status = 'PENDING'
    OR (status = 'RETRY_WAIT' AND next_retry_at <= CURRENT_TIMESTAMP(6))
    OR (status = 'PROCESSING' AND lease_until < CURRENT_TIMESTAMP(6))
  )
ORDER BY requested_at
LIMIT 1
FOR UPDATE SKIP LOCKED;

UPDATE report_transfers
SET status = 'PROCESSING',
    attempt_count = attempt_count + 1,
    lease_owner = :workerId,
    lease_until = DATE_ADD(CURRENT_TIMESTAMP(6), INTERVAL :leaseSeconds SECOND),
    started_at = COALESCE(started_at, CURRENT_TIMESTAMP(6)),
    last_error_code = NULL,
    last_error_message = NULL
WHERE transfer_id = :transferId;

INSERT INTO report_transfer_attempts (
  transfer_id, attempt_no, worker_id
) VALUES (
  :transferId, :attemptNo, :workerId
);

COMMIT;
```

파일 I/O와 LIMS 통신은 DB 락을 해제한 뒤 수행한다. 장시간 전송 시에는 `lease_until` 을 주기적으로 연장한다.

6.2 공유 파일 검증

Spring Boot는 다음 순서로 파일을 읽는다.

1. `storage_key` 를 허용된 로컬 루트에 매핑한다.
2. `root.resolve(package_relative_path).normalize()` 를 계산한다.
3. 계산 결과가 `root.toRealPath()` 하위인지 확인해 경로 탈출을 차단한다.
4. 심볼릭 링크 정책에 따라 링크를 거부하거나 실제 경로를 다시 검증한다.
5. 일반 파일인지 확인한다.
6. `package_size_bytes` 와 실제 크기를 비교한다.
7. SHA-256을 계산해 `package_sha256` 과 비교한다.

8. ZIP central directory를 읽어 유효한 ZIP인지 확인한다.

검증된 ZIP을 현재 위치에서 스트림으로 열어 LIMS에 전송한다. Spring Boot 수신 디렉터리나 임시 영구 저장소로 복사하지 않는다. 전송 라이브러리가 임시 파일을 요구하는 경우에도 작업 종료 즉시 삭제되는 OS 임시 파일만 허용한다.

6.3 성공 처리

LIMS가 성공을 반환하면 같은 트랜잭션에서 현재 attempt와 큐를 완료한다.

```
UPDATE report_transfer_attempts
SET finished_at = CURRENT_TIMESTAMP(6),
    success = TRUE,
    response_code = :responseCode,
    response_message = :responseMessage
WHERE transfer_id = :transferId
AND attempt_no = :attemptNo;

UPDATE report_transfers
SET status = 'COMPLETED',
    completed_at = CURRENT_TIMESTAMP(6),
    external_tracking_id = :trackingId,
    lease_owner = NULL,
    lease_until = NULL,
    next_retry_at = NULL
WHERE transfer_id = :transferId
AND lease_owner = :workerId;
```

6.4 재시도와 최종 실패

네트워크 단절, timeout, LIMS 429, 일시적 5xx는 RETRY_WAIT로 바꾸고 지수 backoff를 적용한다. 잘못된 상대 경로, 크기·SHA 불일치, 손상 ZIP, 인증 실패, 업무키 오류 등 재시도로 해결되지 않는 오류는 즉시 FAILED로 바꾼다.

최대 시도 횟수에 도달한 일시 오류도 FAILED로 전환한다. 모든 시도 결과는 report_transfer_attempts에 남긴다.

7. 보관 및 삭제 정책

- PENDING, PROCESSING, RETRY_WAIT 보고서 ZIP은 삭제하지 않는다.
- COMPLETED, FAILED, CANCELLED 만 보관 기간 정책의 대상이 될 수 있다.
- 파일 삭제 전 report_runs와 전송 이력은 감사 정책에 따라 보존한다.
- 파일이 먼저 사라진 경우 Spring Boot는 REPORT_PACKAGE_NOT_FOUND로 FAILED 처리하고 운영 알림을 남긴다.
- 동일 상대 경로의 파일을 다른 내용으로 덮어쓰지 않는다. 새 보고서는 새 report_id/버전과 새 경로를 사용한다.

8. 제거된 HTTP 계약

다음 항목은 더 이상 사용하지 않는다.

- `POST /api/v1/edge/reports`
- `multipart/form-data` ZIP 전달
- `RIST_SPRING_CALLBACK_URL`
- `RIST_SPRING_CALLBACK_TIMEOUT_SECONDS`
- `RIST_SPRING_CALLBACK_MAX_ATTEMPTS`
- `CALLBACK_PENDING` 작업 상태

실험 PC/C# 프로그램의 기존 Edge 업로드 API에는 변화가 없다. C# 프로그램은 Spring Boot나 전송 큐를 직접 호출하지 않고 `GET /api/v1/jobs/{jobId}` 로 Edge 생성 완료까지만 확인한다.